PharmaLens

Scan the pack. Spot the signs.

OFFICIAL SOFTWARE GUIDE

Localhost & Setup Guide

Understand the platform, why responsible package screening matters, and how to run PharmaLens locally—from guest mode to the complete Supabase-backed environment.

Version 1.0 · October 2026

Preliminary visual screening—not certification

About PharmaLens

PharmaLens is a mobile-first application that helps people examine visible information on pharmaceutical packaging marketed in Nigeria. It combines guided photography, browser-based OCR, user-reviewed fields, and documented reference records.

> **Essential boundary.** An image cannot establish chemical composition, pharmaceutical quality, regulatory approval, authenticity, or whether a medicine is safe to take.

The three outcomes

Reference match

Compared label details are consistent with an available published reference.

Needs review

A visible discrepancy warrants further checking by an appropriate professional.

Unable to verify

The image, extracted evidence, or reference coverage is insufficient.

Core capabilities

Camera-first capture

Rear-camera preference, uploads, rotation, optional panels, compression, and measured quality checks.

Real OCR

Tesseract.js reads visible label text locally in the browser. Users confirm and correct uncertain values.

Transparent screening

Versioned rules compare supported label details without inventing detection scores or verdicts.

Private evidence

Device-only guest history or account-scoped records with private, consent-controlled image retention.

Reference governance

Draft, published, retired, and withdrawn versions preserve provenance and historical evidence.

Review workflow

Explicitly consented requests enter a private reviewer queue with status history and audit events.

Useful signals, responsibly presented

Packaging can reveal useful warning signs—such as inconsistent strength, manufacturer details, expiry information, registration text, or unreadable labelling. PharmaLens helps organise these observations without overstating what a photograph can prove.

1

An accessible first check

A phone camera can help capture and review package information before a user seeks professional advice.

2

Evidence people can inspect

The result shows what was observed, which source was used, and what remains unresolved.

3

Careful result language

The interface never labels a medicine “genuine,” “safe,” or “NAFDAC approved” from an image.

4

Privacy by design

OCR runs locally. Guest records stay on the device. Images upload only after explicit retention consent.

How information moves

Mode	Processing	Storage
Guest	Image correction, quality checks, OCR, and decision rules run in the browser.	Saved scan details remain in browser storage. They are never silently synced after login.
Account	OCR still runs in the browser. Published reference metadata may be queried from Supabase.	Scan evidence can be saved privately. Images require separate upload and retention consent.
Review	The owner explicitly selects a saved scan, describes the concern, and consents.	The request stays private to the owner and authorised reviewers. It is not sent to a regulator.

Professional boundary. PharmaLens supports careful decision-making. It does not replace a licensed pharmacist, laboratory analysis, regulator, or authorised item-authentication service.

Run on localhost in minutes

Quick mode runs the interface, local OCR, guest history, legal pages, and fictional demonstration without cloud credentials.

Before you begin

- ✓ Node.js 22 or newer
- ✓ npm, included with Node.js
- ✓ Git and a current web browser
- ✓ A camera or sample package image for testing

Install and start

```
# Clone the application
git clone https://github.com/Ustaztech/Pharmalens.git
cd Pharmalens

# Install exact locked dependencies
npm ci

# Start the Next.js development server
npm run dev
```

Open **http://localhost:3000**.

> **Guest-mode note.** Do not copy `.env.example` for guest-only use. Placeholder Supabase values count as configured values and cause failed account requests. With no `.env.local`, the app stays in guest mode.

Available immediately

✓ Guest scanning

Capture or upload, run local OCR, correct extracted details, and receive an evidence-limited result.

✓ Device history

Save up to 20 guest records in the current browser. Nothing is uploaded.

✓ Demo mode

Open `/demo` for clearly labelled fictional records, separate from real scans.

— Account features

Authentication, cloud history, reports, and administration remain unavailable until Supabase is configured.

Connect Supabase

1 Create a development project

Create a disposable Supabase project. Copy its project URL and publishable/anonymous key. Never expose a service-role key to the browser.

2 Create the local environment file

```
cp .env.example .env.local
```

Replace every placeholder with the real development-project value:

```
NEXT_PUBLIC_SUPABASE_URL=https://your-real-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-real-anon-key
NEXT_PUBLIC_DEMO_MODE=false
```

3 Apply the database migrations

Run every file in `supabase/migrations/` in numeric order. They create profiles, roles, catalogue versions, scans, private storage, reports, consent, audit records, and row-level policies.

4 Allow the auth callback

Add `http://localhost:3000/auth/callback` to Supabase Auth redirect URLs. Configure an email provider to test verification and password recovery.

5 Restart and create a user

```
npm run dev
```

Environment changes are read when the development server starts.

6 Assign administration safely

New accounts always receive the `user` role. After identity verification, assign the first administrator directly in PostgreSQL:

```
update public.profiles
set role = 'admin'
where id = '<verified-user-uuid>';
```

Use and verify the application

Complete a local scan

1. Open `/scan` and use the rear camera or choose an image.
2. Keep the entire front panel visible in bright, even light.
3. Add back or side panels when they contain batch, expiry, or manufacturer evidence.
4. Review OCR text and correct uncertain values.
5. Select **Screen label details** and read the evidence scope and expiry status.
6. Save to device-only or private account history when appropriate.

Verification commands

```
npm test
npm run lint
npm run build
npm run test:e2e
```

If Playwright reports that Chromium is missing:

```
npx playwright install chromium
npm run test:e2e
```

Reference governance

Draft first

New product reference versions begin as drafts and require provenance and review.

Published only

Only published versions participate in normal screening.

History preserved

Updates and withdrawals do not rewrite the reference snapshot attached to an earlier scan.

No copied proof

A registration number, barcode, or QR code is not independent evidence of authenticity.

> **Current verification.** The repository passes TypeScript production build, ESLint, 17 Vitest tests, and 48 Playwright checks across 360, 390, 768, and 1440 px layouts.

Troubleshooting

Problem	What to do
Account services are unavailable	Use real Supabase values in `.env.local` and restart. For guest mode, remove the file rather than keeping placeholders.
Camera access is denied	Grant browser permission, close competing camera apps, or choose an image. Non-local deployments require HTTPS.
First OCR scan is slow	Tesseract.js loads OCR resources in the browser. Keep the tab open and use a sharp, well-lit image.
Readable product cannot be verified	A published reference or required field may be missing. PharmaLens correctly returns “Unable to verify” when evidence is insufficient.
Verification email is missing	Check the Supabase email provider, templates, rate limits, redirect URLs, and spam folder.
Port 3000 is occupied	Run `npm run dev -- -p 3001`, then add the matching auth callback URL if needed.

Before a public launch

- ✓ Provide attributable, reviewed reference data and authorised reviewers.
- ✓ Define retention periods, operator identity, support contacts, and operating procedures.
- ✓ Configure monitoring, backups, rate limits, and incident response.
- ✓ Complete privacy, security, accessibility, legal, and regulatory review.
- ✓ Validate physical-device camera behaviour across supported mobile browsers.
- ✓ Independently validate any medicine-authenticity claims or future visual model.

> **A successful build is not clinical validation.** PharmaLens must not be described as a validated counterfeit detector or as proof that a medicine is genuine, approved, or safe.

Technical references

See `README.md` and `docs/ARCHITECTURE.md`, `SCREENING_POLICY.md`, `PRIVACY_AND_DATA_FLOW.md`, `DEPLOYMENT.md`, `EXTERNAL_DEPENDENCIES.md`, and `TEST_RESULTS.md`.